## Module 3

## Relational Model and relational Algebra

Introduction to the Relational Model, relational schema and concept of keys.

Mapping the ER and EER Model to the Relational Model, Relational

Algebra-operators. Relational Algebra Queries.

### Introduction to the Relational Model

The relational Model was proposed by E.F. Codd to model data in the form of relations or tables. After designing the conceptual model of the Database using ER diagram, we need to convert the conceptual model into a relational model which can be implemented using any RDBMS language like Oracle SQL, MySQL, etc. So we will see what the Relational Model is.

**What is the Relational Model?**
The relational model represents how data is stored in Relational Databases. A relational database stores data in the form of relations (tables). Consider a relation STUDENT with attributes ROLL_NO, NAME, ADDRESS, PHONE, and AGE shown in Table 1.

**STUDENT**

| ROLL_NO | NAME | ADDRESS | PHONE | AGE |
|---|---|---|---|---|
| 1 | RAM | DELHI | 9455123451 | 18 |
| 2 | RAMESH | GURGAON | 9652431543 | 18 |
| 3 | SUJIT | ROHTAK | 9156253131 | 20 |
| 4 | SURESH | DELHI | | 18 |

**IMPORTANT TERMINOLOGIES**

- **Attribute:** Attributes are the properties that define a relation. e.g.; **ROLL_NO**, **NAME**
- **Relation Schema:** A relation schema represents the name of the relation with its attributes. e.g.; STUDENT (ROLL_NO, NAME, ADDRESS, PHONE, and AGE) is the relation schema for STUDENT. If a schema has more than 1 relation, it is called Relational Schema.
- **Tuple:** Each row in the relation is known as a tuple. The above relation contains 4 tuples, one of which is shown as:

1 RAM DELHI 9455123451 18

- **Relation Instance:** The set of tuples of a relation at a particular instance of time is called a relation instance. Table 1 shows the relation instance of STUDENT at a particular time. It can change whenever there is an insertion, deletion, or update in the database.
- **Degree:** The number of attributes in the relation is known as the degree of the relation. The **STUDENT** relation defined above has degree 5.
- **Cardinality:** The number of tuples in a relation is known as cardinality. The **STUDENT** relation defined above has cardinality 4.
- **Column:** The column represents the set of values for a particular attribute. The column **ROLL_NO** is extracted from the relation STUDENT.

**ROLL_NO**

1

2

3

4

- **NULL Values:** The value which is not known or unavailable is called a NULL value. It is represented by blank space. e.g.; PHONE of STUDENT having ROLL_NO 4 is NULL.

**Relational schema**

Relation schema defines the design and structure of the relation like it consists of the relation name, set of attributes/field names/column names. every attribute would have an associated domain.

There is a student named Geeks, she is pursuing B.Tech, in the 4th year, and belongs to IT department (department no. 1) and has roll number 1601347 She is proctored by Mrs. S Mohanty. If we want to represent this using databases we would have to create a student table with name, sex, degree, year, department, department number, roll number and proctor (adviser) as the attributes.

student (rollNo, name, degree, year, sex, deptNo, advisor)

**Note –**
If we create a database, details of other students can also be recorded.
Similarly, we have the IT Department, with department Id 1, having Mrs. Sujata Chakravarty as the head of department. And we can call the department on the number 0657 228662 .

This and other departments can be represented by the department table, having department ID, name, hod and phone as attributes.

department (deptId, name, hod, phone)

The course that a student has selected has a courseid, course name, credit and department number.

course (coursId, ename, credits, deptNo)

The professor would have an employee Id, name, sex, department no. and phone number.

professor (empId, name, sex, startYear, deptNo, phone)

We can have another table named enrollment, which has roll no, courseId, semester, year and grade as the attributes.

enrollment (rollNo, coursId, sem, year, grade)

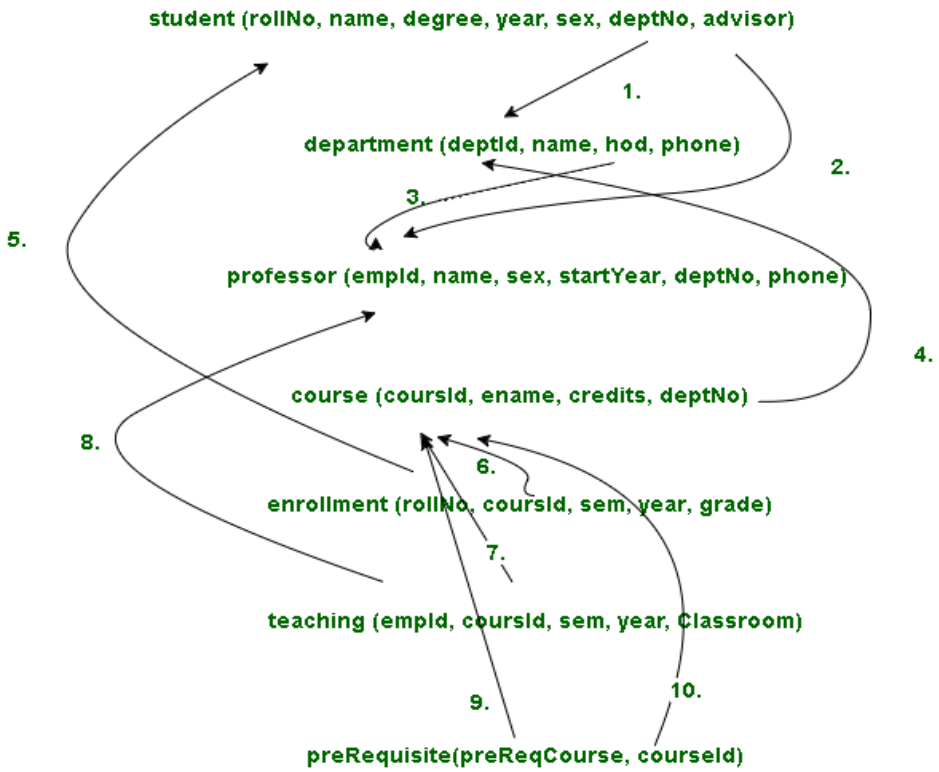
Teaching can be another table, having employee id, course id, semester, year and classroom as attributes.

teaching (empId, coursed, sem, year, Classroom)

When we start courses, there are some courses which another course that needs to be completed before starting the current course, so this can be represented by the Prerequisite table having prerequisite course and course id attributes.

prerequisite (preReqCourse, courseId)

The relations between them is represented through arrows in the following **Relation diagram**,




1. This represents that the deptNo in student table table is same as deptId used in department table. deptNo in student table is a foreign key. It refers to deptId in department table.

2. This represents that the advisor in student table is a foreign key. It refers to empId in professor table.

3. This represents that the hod in department table is a foreign key. It refers to empId in professor table.

4. This represents that the deptNo in course table table is same as deptId used in department table. deptNo in student table is a foreign key. It refers to deptId in department table.

5. This represents that the rollNo in enrollment table is same as rollNo used in student table.

6. This represents that the courseId in enrollment table is same as courseId used in course table.

7. This represents that the courseId in teaching table is same as courseId used in course table.

8. This represents that the empId in teaching table is same as empId used in professor table.

9. This represents that preReqCourse in prerequisite table is a foreign key. It refers to courseId in course table.

10. This represents that the deptNo in student table is same as deptId used in department table.

**Note –**
startYear in professor table is same as year in student table

**Concept of keys:**

**Different Types of Keys in Relational Model**
**Candidate Key:** The minimal set of attributes that can uniquely identify a tuple is known as a candidate key. For Example, STUD_NO in STUDENT relation.

- It is a minimal super key.
- It is a super key with no repeated data is called a candidate key.
- The minimal set of attributes that can uniquely identify a record.
- It must contain unique values.
- It can contain NULL values.
- Every table must have at least a single candidate key.
- A table can have multiple candidate keys but only one primary key (the primary key cannot have a NULL value, so the candidate key with NULL value can't be the primary key).

Eg: Studid, Roll No, and email are candidate keys.

- The value of the Candidate Key is unique and may be null for tuple.
- There can be more than one candidate key in a relation. For Example, STUD_NO is the candidate key for relation STUDENT.
- The candidate key can be simple (having only one attribute) or composite as well. For Example, {STUD_NO, COURSE_NO} is a composite candidate key for relation STUDENT_COURSE.
- No, of candidate keys in a Relation are nC(floor(n/2)),for example if a Relation have 5 attributes i.e. R(A,B,C,D,E) then total no of candidate keys are 5C(floor(5/2))=10.

***Note:*** *In SQL Server a unique constraint that has a nullable column,* ***allows*** *the value '****null****' in that column* ***only once****. That's why the STUD_PHONE attribute is a candidate here, but can not be 'null' value in the primary key attribute.*

**Super Key:** The set of attributes that can uniquely identify a tuple is known as Super Key. For Example, STUD_NO, (STUD_NO, STUD_NAME), etc.
Or A super key is a group of single or multiple keys that identifies rows in a table. It supports NULL values. Example: SNO+PHONE is a super key.

- Adding zero or more attributes to the candidate key generates the super key.
- A candidate key is a super key but vice versa is not true.

**Primary Key:** There can be more than one candidate key in relation out of which one can be chosen as the primary key. For Example, STUD_NO, as well as STUD_PHONE, are candidate keys for relation STUDENT but STUD_NO can be chosen as the primary key (only one out of many candidate keys).

**Primary Key:**

- It is a unique key.
- It can identify only one tuple (a record) at a time.
- It has no duplicate values, it has unique values.
- It cannot be NULL.
- Primary keys are not necessarily to be a single column; more than one the column can also be a primary key for a table.
- Eg:- STUDENT table SNO is a primary key
  SNO SNAME ADDRESS PHONE

**Alternate Key:** The candidate key other than the primary key is called an alternate key. For Example, STUD_NO, as well as STUD_PHONE both, are candidate keys for relation STUDENT but STUD_PHONE will be an alternate key (only one out of many candidate keys). It is a secondary key

- All the keys which are not primary keys are called alternate keys.
- It contains two or more fields to identify two or more records.
- These values are repeated.
  Eg:- SNAME, ADDRESS is Alternate keys

**Foreign Key:** If an attribute can only take the values which are present as values of some other attribute, it will be a foreign key to the attribute to which it refers. The relation which is being referenced is called referenced relation and the corresponding attribute is called referenced attribute and the relation which refers to the referenced relation is called referencing relation and the corresponding attribute is called referencing attribute. The referenced attribute of the referenced relation should be the primary key to it. For Example, STUD_NO in STUDENT_COURSE is a foreign key to STUD_NO in STUDENT relation.

- It is a key it acts as a primary key in one table and it acts as secondary key in another table.
- It combines two or more relations (table) at a time.
- They act as a cross-reference between the tables.
- For example, DNO is a primary key in the DEPT table and a non-key in EMP

It may be worth noting that unlike the Primary Key of any given relation, Foreign Key can be NULL as well as may contain duplicate tuples i.e. it need not follow uniqueness constraint. For Example, STUD_NO in STUDENT_COURSE relation is not unique. It has been repeated for the first and third tuples. However, the STUD_NO in STUDENT relation is a primary key and it needs to be always unique, and it cannot be null.

**Composite Key:** Sometimes, a table might not have a single column/attribute that uniquely identifies all the records of a table. In order to uniquely identify rows of a table, combination of two or more columns/attributes can be used. For example, FULLNAME + DOB can be combined together to access the details of a student. It still can give duplicate values in rare cases. So , we need to find the optimal set of attributes that can uniquely identify rows in a table.

- It acts as a primary key if there is no primary key in a table
- Two or more attributes are used to together to make a composite key.
- Different combinations of attributes may give different accuracy in terms of identifying the rows uniquely.

**Mapping ER and EER diagrams into relational schemas**

First I'll tell you how to map the ER diagram into a relational schema.

**Mapping ER diagrams into relational schemas**

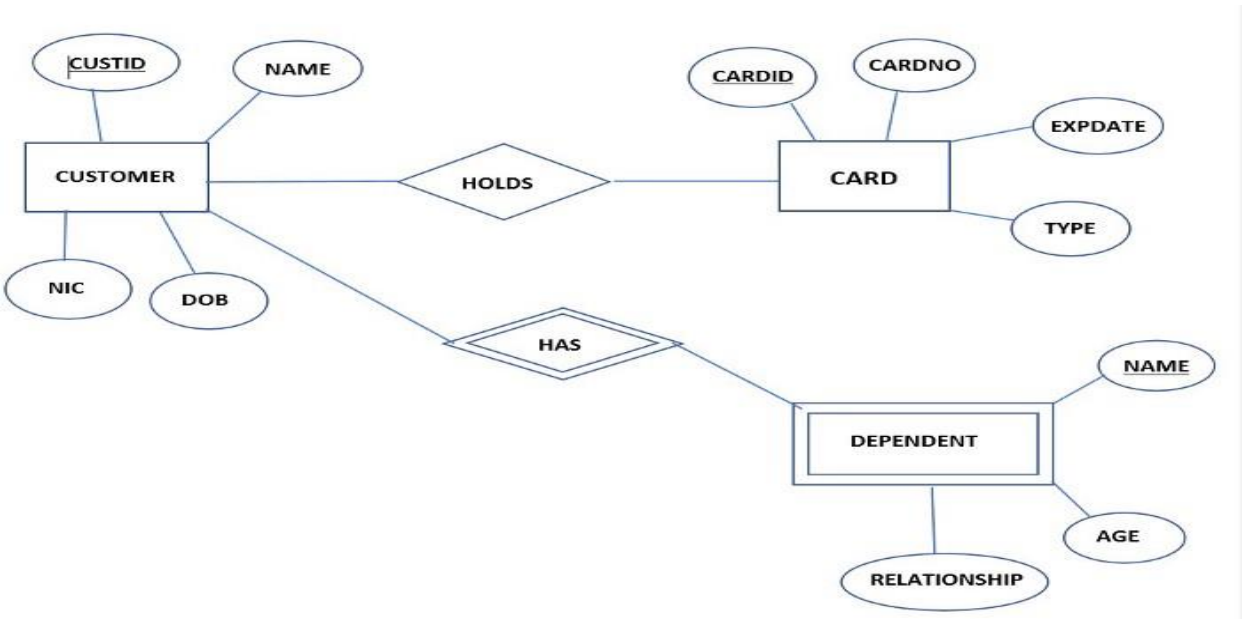
Follow the steps one by one to get it done.

1. ***Mapping strong entities.***
2. ***Mapping weak entities.***
3. ***Map binary one-to-one relations.***
4. ***Map binary one-to-many relations***
5. ***Map binary many-to-many relations.***
6. ***Map multivalued attributes.***
7. ***Map N-ary relations***

Let's go deep with the examples.

***1. Mapping strong entities.***

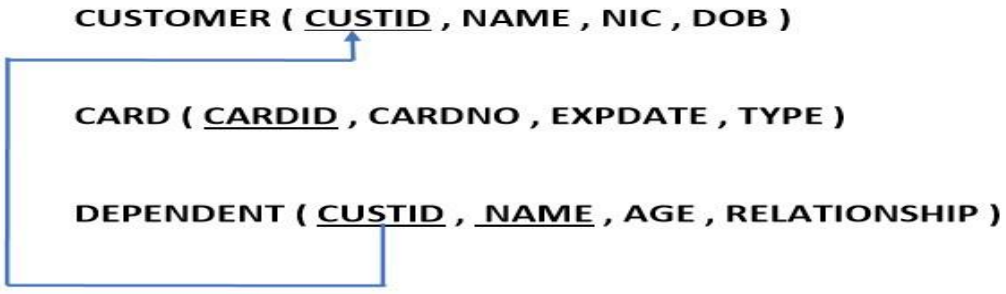
***2. Mapping weak entities.***




Above it shows an ER diagram with its relationships. You can see there are two strong entities with relationships and a weak entity with a weak relationship.

When you going to make a relational schema first you have to identify all entities with their attributes. You have to write attributes in brackets as shown below. Definitely you have to underline the primary keys. In the above **DEPENDENT** is a weak entity. To make it strong go through the weak relationship and identify the entity which connects with this. Then you have written that entity's primary key inside the weak entity bracket.

Then you have to map the primary key to where you took from as shown below.

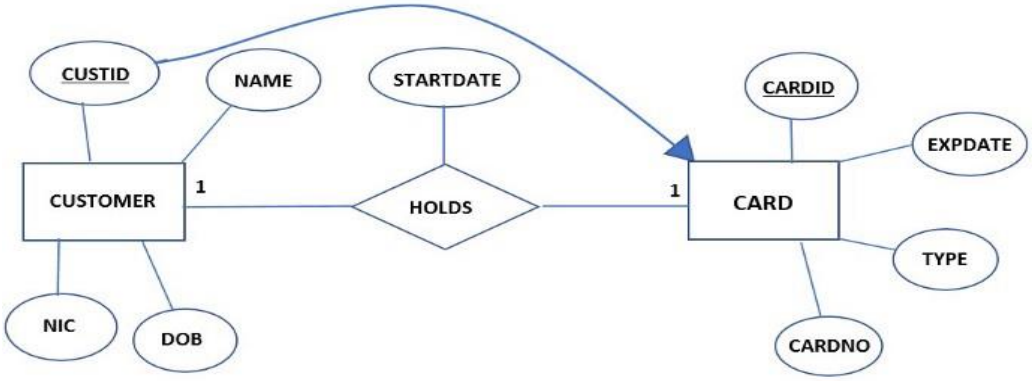


***3. Map binary one to one relations.***

Let's assume that the relationship between CUSTOMER and CARD is one to one.

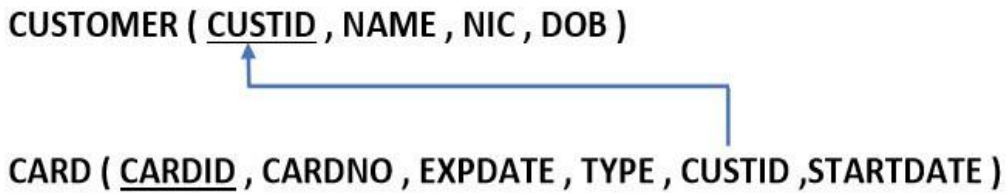
There are three occasions where one to one relations take place according to the participation constraints.

I. Both sides have partial participation.



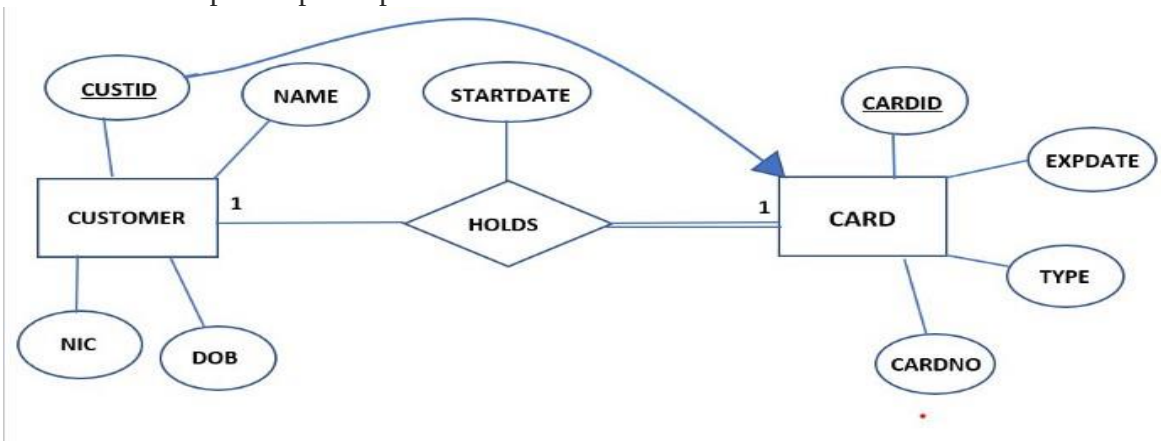

When both sides have partial participation you can send any of the entity's primary key to others. At the same time, if there are attributes in the relationship between those two entities, it is also sent to other entity as shown above.

So, now let us see how we write the relational schema.




Here you can see I have written **CUSTID** and **STARTDATE** inside the **CARD** table. Now you have to map **CUSTID** from where it comes. That's it.□

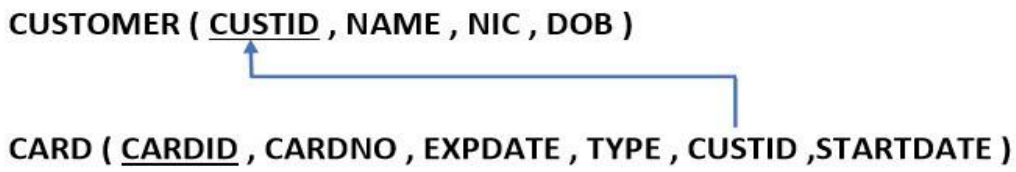
II. One side has partial participation.



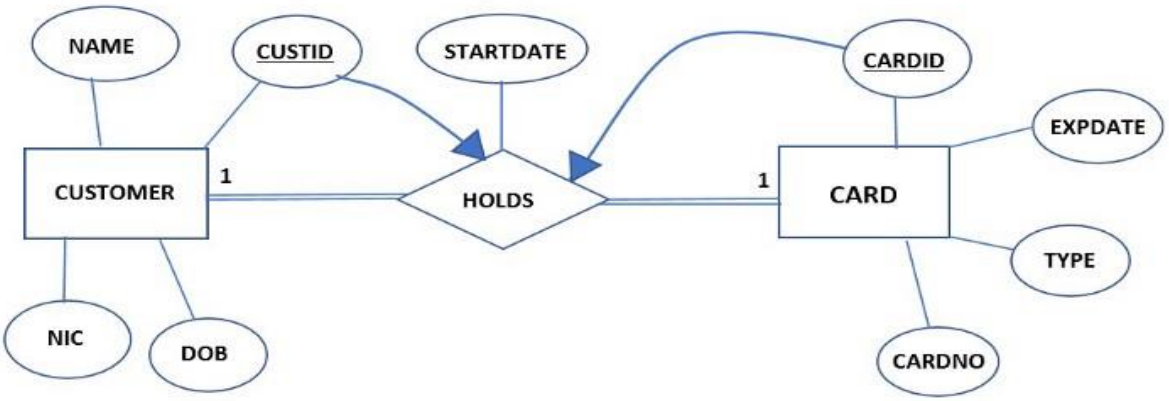

You can see between the relationship and **CARD** entity it has total participation.

When there is total participation definitely the primary of others comes to this. And also if there are attributes in the relationship it also comes to total participation side.

Then you have to map them as below.




III. Both sides have total participation



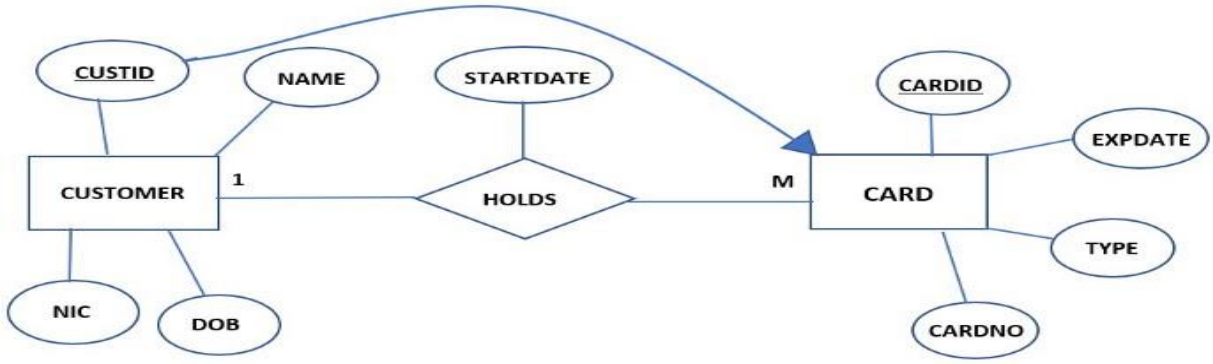

If both sides have total participation you need to make a new relationship with a suitable name and merge entities and the relationship.

Following it shows how we should write the relation.

CUST_HOLD( CUSTID , NAME , NIC , DOB , CARDID , CARDNO , EXPDATE , TYPE ,STARTDATE )

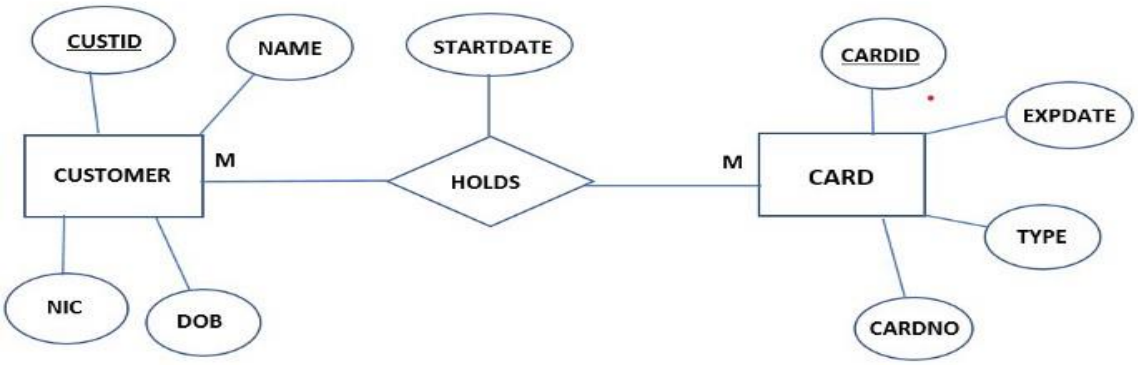
Now let us see how to map one to many relations.

***4. Map binary one-to-many relations***



If it is one-to-many, always to the many side other entities' primary keys and the attributes in the relationship go to the *many* side. No matter about participation. And then you have to map the primary key.

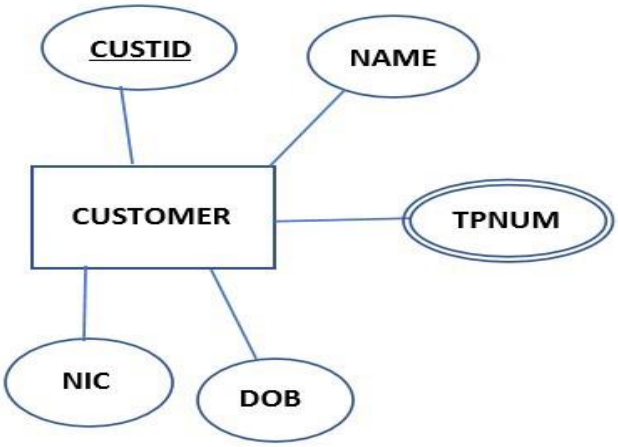

***5. Map binary many to many relations.***



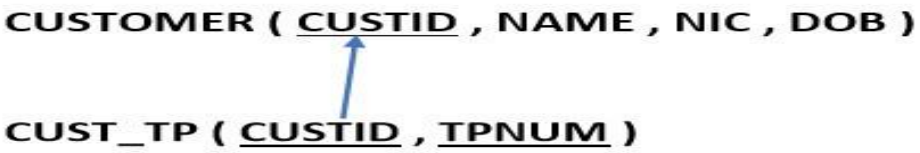

If it is many to many relations you should always make a new relationship with the name of the relationship between the entities.

And there you should write both primary keys of the entities and attributes in the relationship and map them to the initials as shown below.


***6. Map multivalued attributes.***




If there are multivalued attributes you have to make a new relationship with a suitable name and write the primary key of the entity which belongs to the multivalued attribute and also the name of the multivalued attribute as shown below.

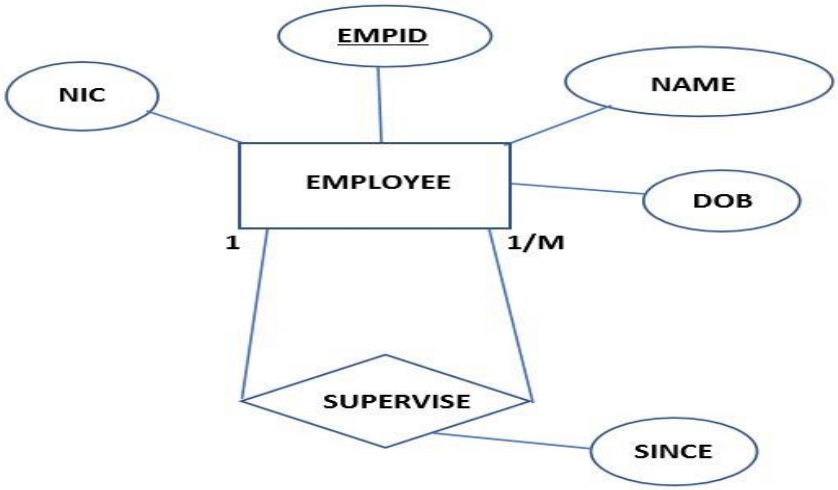



***7. Map N-ary relations.***

First, let us consider unary relationships.

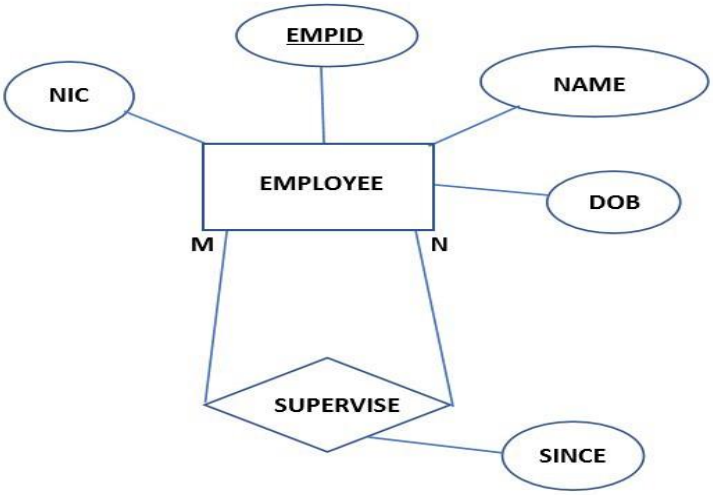
We categorized them into two.

I. one-to-one and one to many relations.

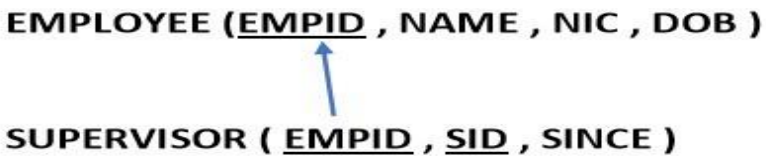



If it is unary and one to one or one to many relations you do not need to make a new relationship you just want to add a new primary key to the current entity as shown below and map it to the initial. For example, in the above diagram, the employee is supervised by the supervisor. Therefore we need to make a new primary key as **SID** and map it to **EMPID**. Because of **SID** also an **EMPID**.

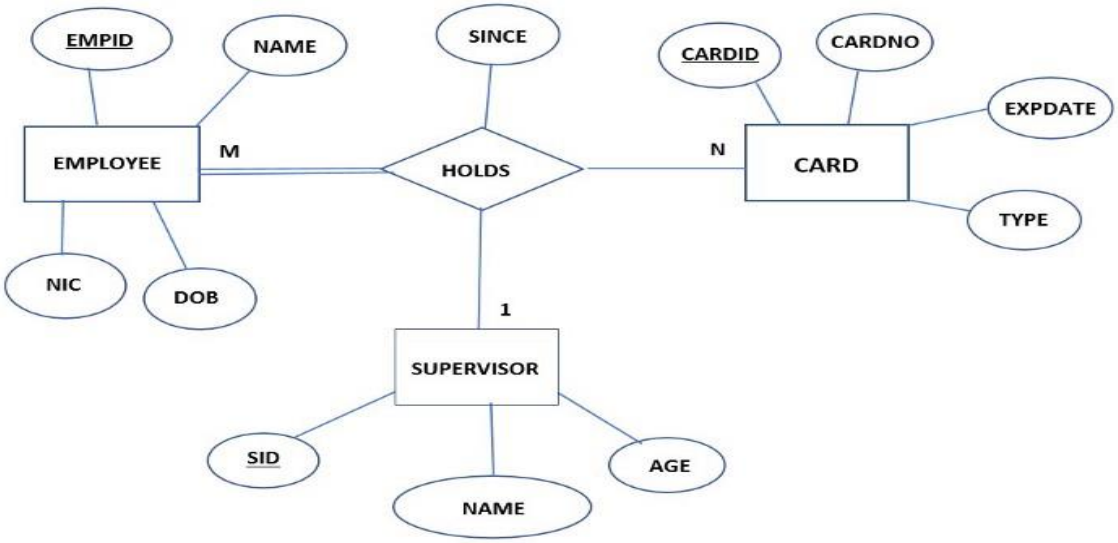

II. many-to-many relations.



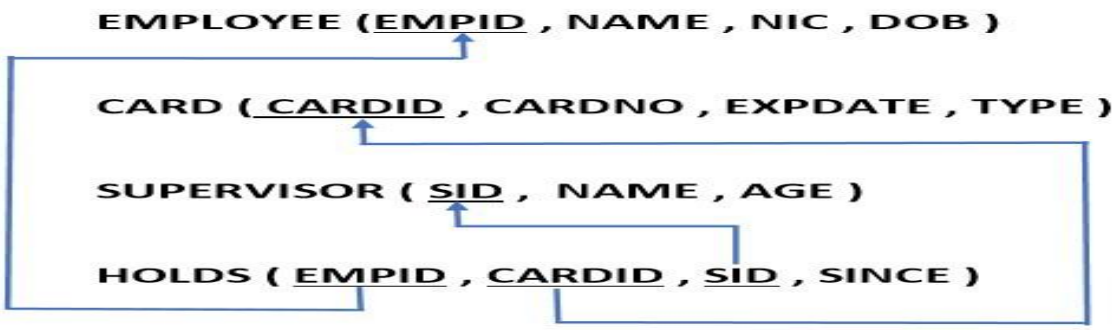
If it is unary and many to many relations you need to make a new relationship with a suitable name. Then you have to give it a proper primary key and it should map to where it comes as shown below.




Now let us see how to map relations with more than two entities.

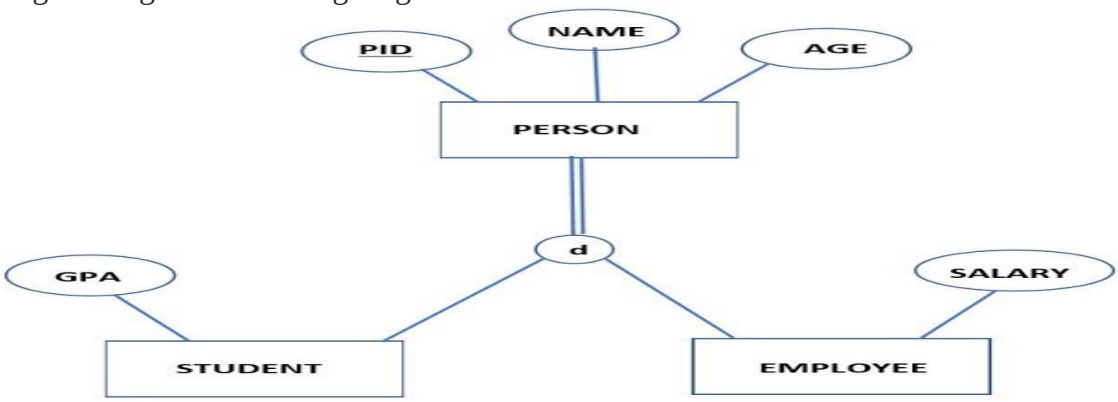



**If there are more than three entities for a relationship you have to make a new relation table and put all primary keys of connected entities and all attributes in the relationship. And in the end, you have to map them as shown below.**



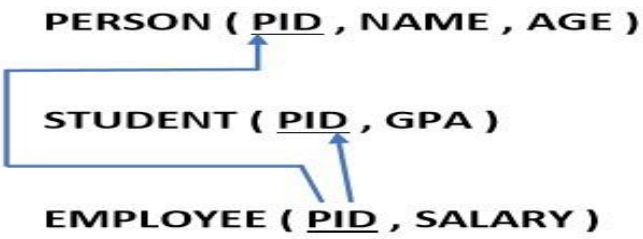

**Mapping EER diagrams into relational schemas.**

Let us go through the following diagram.




There are four ways to draw relational schema for an EER. You have to choose the most suitable one. In the end, I'll give you the guidelines on how to choose the best and most suitable way.

**First method**



Here we write separate relations to all superclass entities and subclass entities. And here we have to write the superclass entities' primary key to all subclass entities and then map them as shown above. Note that we write only the attributes belongs to each entity.

**Second method**

STUDENT ( PID , GPA , NAME , AGE )

EMPLOYEE ( PID , SALARY , NAME , AGE )

Here we do not write the superclass entity but in each subclass entity, we write all attributes that are in superclass entity.

**Third method**

PERSON ( PID , NAME , AGE , SALARY , GPA , PERSONTYPE )

Here we write only the superclass entity and write all the attributes which belong to subclass entities. Specialty in here is that to identify that **PERSON** is an **EMPLOYEE** or **STUDENT** we add a column as **PERSONTYPE**. After the table creates we can mark as a **STUDENT** or **EMPLOYEE**.

**Fourth method**

PERSON ( PID , NAME , AGE , SALARY , GPA , STUDENT , EMPLOYEE )

Here instead of PERSONTYPE, we write STUDENT and EMPLOYEE both.

The reason for that is sometime PERSON will belong to both categories.

Somewhat confusing right?Don't worry read the following guidelines to clear out them.

Now let us see how to select the best and most suitable method to write the relational schema.

**Guidelines**

*1.If sub-entities have more attributes (local or foreign attributes)*

Select the **first or second method**.

From this two,

If EER is totally specialized -> select the **second method**.

If EER is partially specialized -> select **first method**.

*2. If sub-entities have fewer attributes (local or foreign attributes)*

Select the **third or fourth method**.

From this two,

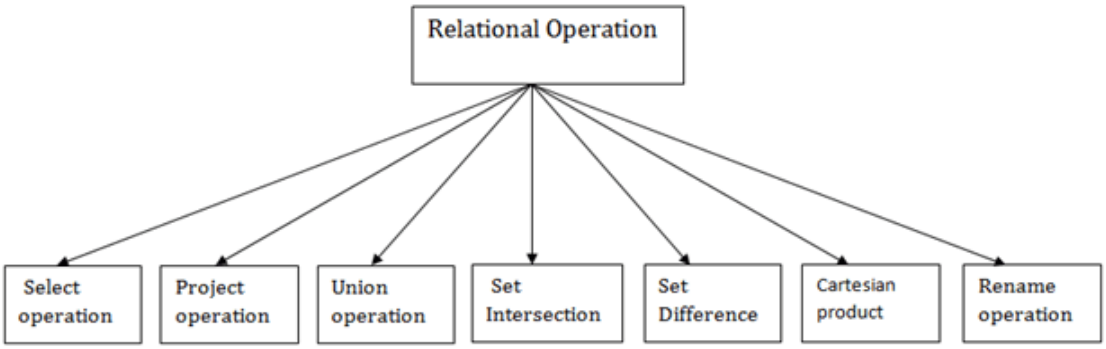
If EER is disjoint-> select the **third method**.

If EER is overlap -> select **forth method**.

## Relational Algebra

Relational algebra is a procedural query language. It gives a step by step process to obtain the result of the query. It uses operators to perform queries.

## Types of Relational operation




### 1. Select Operation:

- The select operation selects tuples that satisfy a given predicate.
- It is denoted by sigma (σ).

1. Notation: σ p(r)

**Where:**

**σ** is used for selection prediction
**r** is used for relation
**p** is used as a propositional logic formula which may use connectors like: AND OR and NOT. These relational can use as relational operators like =, ≠, ≥, <, >, ≤.

**For example: LOAN Relation**

| BRANCH_NAME | LOAN_NO | AMOUNT |
|---|---|---|

| | | |
|---|---|---|
| Downtown | L-17 | 1000 |
| Redwood | L-23 | 2000 |
| Perryride | L-15 | 1500 |
| Downtown | L-14 | 1500 |
| Mianus | L-13 | 500 |
| Roundhill | L-11 | 900 |
| Perryride | L-16 | 1300 |

**Input:**

1. σ BRANCH_NAME="perryride" (LOAN)

**Output:**

| BRANCH_NAME | LOAN_NO | AMOUNT |
|---|---|---|
| Perryride | L-15 | 1500 |
| Perryride | L-16 | 1300 |

2. Project Operation:

- This operation shows the list of those attributes that we wish to appear in the result. Rest of the attributes are eliminated from the table.
- It is denoted by ∏.

1. Notation: ∏ A1, A2, An (r)

**Where**

**A1**, **A2**, **A3** is used as an attribute name of relation **r**.

**Example: CUSTOMER RELATION**

| NAME | STREET | CITY |
|---|---|---|
| Jones | Main | Harrison |
| Smith | North | Rye |
| Hays | Main | Harrison |
| Curry | North | Rye |
| Johnson | Alma | Brooklyn |
| Brooks | Senator | Brooklyn |

**Input:**

1. ∏ NAME, CITY (CUSTOMER)

**Output:**

| NAME | CITY |
|---|---|
| Jones | Harrison |
| Smith | Rye |
| Hays | Harrison |

| | |
|---|---|
| Curry | Rye |
| Johnson | Brooklyn |
| Brooks | Brooklyn |

### 3. Union Operation:

- Suppose there are two tuples R and S. The union operation contains all the tuples that are either in R or S or both in R & S.
- It eliminates the duplicate tuples. It is denoted by ∪.

1. Notation: R ∪ S

A union operation must hold the following condition:

- R and S must have the attribute of the same number.
- Duplicate tuples are eliminated automatically.

Example:

**DEPOSITOR RELATION**

| **CUSTOMER_NAME** | **ACCOUNT_NO** |
|---|---|
| Johnson | A-101 |
| Smith | A-121 |
| Mayes | A-321 |
| Turner | A-176 |
| Johnson | A-273 |

| | |
|---|---|
| Jones | A-472 |
| Lindsay | A-284 |

**BORROW RELATION**

| CUSTOMER_NAME | LOAN_NO |
|---|---|
| Jones | L-17 |
| Smith | L-23 |
| Hayes | L-15 |
| Jackson | L-14 |
| Curry | L-93 |
| Smith | L-11 |
| Williams | L-17 |

**Input:**

1. ∏ CUSTOMER_NAME (BORROW) ∪ ∏ CUSTOMER_NAME (DEPOSITOR)

**Output:**

| CUSTOMER_NAME |
|---|
| Johnson |

| |
|---|
| Smith |
| Hayes |
| Turner |
| Jones |
| Lindsay |
| Jackson |
| Curry |
| Williams |
| Mayes |

4. Set Intersection:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in both R & S.
- It is denoted by intersection ∩.

1. Notation: R ∩ S

**Example:** Using the above DEPOSITOR table and BORROW table

**Input:**

1. ∏ CUSTOMER_NAME (BORROW) ∩ ∏ CUSTOMER_NAME (DEPOSITOR)

**Output:**

| **CUSTOMER_NAME** |
|---|

| |
|---|
| Smith |
| Jones |

### 5. Set Difference:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in R but not in S.
- It is denoted by intersection minus (-).

1. Notation: R - S

**Example:** Using the above DEPOSITOR table and BORROW table

**Input:**

1. ∏ CUSTOMER_NAME (BORROW) - ∏ CUSTOMER_NAME (DEPOSITOR)

**Output:**

| **CUSTOMER_NAME** |
|---|
| Jackson |
| Hayes |
| Willians |
| Curry |

### 6. Cartesian product

- The Cartesian product is used to combine each row in one table with each row in the other table. It is also known as a cross product.
- It is denoted by X.

1. Notation: E X D

   Example:

   **EMPLOYEE**

| EMP_ID | EMP_NAME | EMP_DEPT |
|---|---|---|
| 1 | Smith | A |
| 2 | Harry | C |
| 3 | John | B |

   **DEPARTMENT**

| DEPT_NO | DEPT_NAME |
|---|---|
| A | Marketing |
| B | Sales |
| C | Legal |

   **Input:**

1. EMPLOYEE X DEPARTMENT

   **Output:**

| EMP_ID | EMP_NAME | EMP_DEPT | DEPT_NO | DEPT_NAME |
|---|---|---|---|---|
| 1 | Smith | A | A | Marketing |

| | | | | |
|---|---|---|---|---|
| 1 | Smith | A | B | Sales |
| 1 | Smith | A | C | Legal |
| 2 | Harry | C | A | Marketing |
| 2 | Harry | C | B | Sales |
| 2 | Harry | C | C | Legal |
| 3 | John | B | A | Marketing |
| 3 | John | B | B | Sales |
| 3 | John | B | C | Legal |

### 7. Rename Operation:

The rename operation is used to rename the output relation. It is denoted by **rho** (ρ).

**Example:** We can use the rename operator to rename STUDENT relation to STUDENT1.

1. ρ(STUDENT1, STUDENT)

*Note: Apart from these common operations Relational algebra can be used in Join operations.*

### Join Operations:

A Join operation combines related tuples from different relations, if and only if a given join condition is satisfied. It is denoted by ⋈.

### Example:

**EMPLOYEE**

| **EMP_CODE** | **EMP_NAME** |
|---|---|

| | |
|---|---|
| 101 | Stephan |
| 102 | Jack |
| 103 | Harry |

**SALARY**

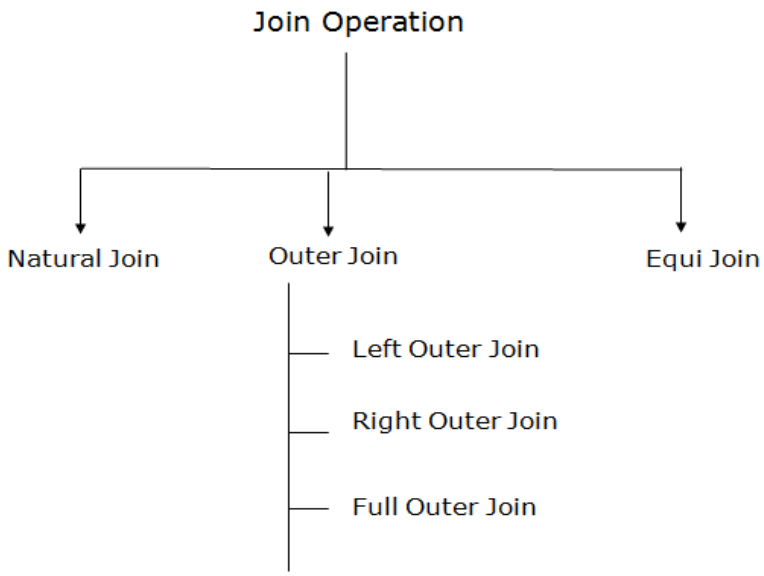
| EMP_CODE | SALARY |
|---|---|
| 101 | 50000 |
| 102 | 30000 |
| 103 | 25000 |

1. Operation: (EMPLOYEE ⋈ SALARY)

**Result:**

| EMP_CODE | EMP_NAME | SALARY |
|---|---|---|
| 101 | Stephan | 50000 |
| 102 | Jack | 30000 |
| 103 | Harry | 25000 |

Types of Join operations:




1. Natural Join:

- A natural join is the set of tuples of all combinations in R and S that are equal on their common attribute names.
- It is denoted by ⋈.

**Example:** Let's use the above EMPLOYEE table and SALARY table:

**Input:**

1. ∏EMP_NAME, SALARY (EMPLOYEE ⋈ SALARY)

**Output:**

| EMP_NAME | SALARY |
|---|---|
| Stephan | 50000 |
| Jack | 30000 |
| Harry | 25000 |

## 2. Outer Join:

The outer join operation is an extension of the join operation. It is used to deal with missing information.

**Example:**

**EMPLOYEE**

| EMP_NAME | STREET | CITY |
|---|---|---|
| Ram | Civil line | Mumbai |
| Shyam | Park street | Kolkata |
| Ravi | M.G. Street | Delhi |
| Hari | Nehru nagar | Hyderabad |

**FACT_WORKERS**

| EMP_NAME | BRANCH | SALARY |
|---|---|---|
| Ram | Infosys | 10000 |
| Shyam | Wipro | 20000 |
| Kuber | HCL | 30000 |
| Hari | TCS | 50000 |

**Input:**

1. (EMPLOYEE ⋈ FACT_WORKERS)

**Output:**

| EMP_NAME | STREET | CITY | BRANCH | SALARY |
|---|---|---|---|---|
| Ram | Civil line | Mumbai | Infosys | 10000 |
| Shyam | Park street | Kolkata | Wipro | 20000 |
| Hari | Nehru nagar | Hyderabad | TCS | 50000 |

An outer join is basically of three types:

a. Left outer join
b. Right outer join
c. Full outer join

a. Left outer join:

- Left outer join contains the set of tuples of all combinations in R and S that are equal on their common attribute names.
- In the left outer join, tuples in R have no matching tuples in S.
- It is denoted by ⟕.

**Example:** Using the above EMPLOYEE table and FACT_WORKERS table

**Input:**

1. EMPLOYEE ⟕ FACT_WORKERS

| EMP_NAME | STREET | CITY | BRANCH | SALARY |
|---|---|---|---|---|
| Ram | Civil line | Mumbai | Infosys | 10000 |
| Shyam | Park street | Kolkata | Wipro | 20000 |

| | | | | |
|---|---|---|---|---|
| Hari | Nehru street | Hyderabad | TCS | 50000 |
| Ravi | M.G. Street | Delhi | NULL | NULL |

b. Right outer join:

- Right outer join contains the set of tuples of all combinations in R and S that are equal on their common attribute names.
- In right outer join, tuples in S have no matching tuples in R.
- It is denoted by ⋈.

**Example:** Using the above EMPLOYEE table and FACT_WORKERS Relation

**Input:**

1. EMPLOYEE ⋈ FACT_WORKERS

**Output:**

| EMP_NAME | BRANCH | SALARY | STREET | CITY |
|---|---|---|---|---|
| Ram | Infosys | 10000 | Civil line | Mumbai |
| Shyam | Wipro | 20000 | Park street | Kolkata |
| Hari | TCS | 50000 | Nehru street | Hyderabad |
| Kuber | HCL | 30000 | NULL | NULL |

c. Full outer join:

- Full outer join is like a left or right join except that it contains all rows from both tables.
- In full outer join, tuples in R that have no matching tuples in S and tuples in S that have no matching tuples in R in their common attribute name.
- It is denoted by ⟗.

**Example:** Using the above EMPLOYEE table and FACT_WORKERS table

**Input:**

1. EMPLOYEE ⋈ FACT_WORKERS

**Output:**

| EMP_NAME | STREET | CITY | BRANCH | SALARY |
|---|---|---|---|---|
| Ram | Civil line | Mumbai | Infosys | 10000 |
| Shyam | Park street | Kolkata | Wipro | 20000 |
| Hari | Nehru street | Hyderabad | TCS | 50000 |
| Ravi | M.G. Street | Delhi | NULL | NULL |
| Kuber | NULL | NULL | HCL | 30000 |

### 3. Equi join:

It is also known as an inner join. It is the most common join. It is based on matched data as per the equality condition. The equi join uses the comparison operator(=).

**Example:**

**CUSTOMER RELATION**

| CLASS_ID | NAME |
|---|---|
| 1 | John |
| 2 | Harry |

| | |
|---|---|
| 3 | Jackson |

**PRODUCT**

| PRODUCT_ID | CITY |
|---|---|
| 1 | Delhi |
| 2 | Mumbai |
| 3 | Noida |

**Input:**

1. CUSTOMER ⋈ PRODUCT

**Output:**

| CLASS_ID | NAME | PRODUCT_ID | CITY |
|---|---|---|---|
| 1 | John | 1 | Delhi |
| 2 | Harry | 2 | Mumbai |
| 3 | Harry | 3 | Noida |

**Relational Algebra Queries:**

Given below are a few examples of a database and a few queries based on that.

**(1).** Suppose there is a banking database which comprises following tables :
Customer(Cust_name, Cust_street, Cust_city)

Branch(Branch_name, Branch_city, Assets)

Account (Branch_name, Account_number, Balance)

Loan(Branch_name, Loan_number, Amount)

Depositor(Cust_name, Account_number)

Borrower(Cust_name, Loan_number)

**Query 1:** Find the names of all the customers who have taken a loan from the bank and also have an account at the bank.
Solution:

**Step 1 :** Identify the relations that would be required to frame the resultant query.
First half of the query(i.e. names of customers who have taken loan) indicates "borrowers" information.

So Relation 1 ——> Borrower.

Second half of the query needs Customer Name and Account number which can be obtained from Depositor relation.

Hence, Relation 2——> Depositor.

**Step 2 :** Identify the columns which you require from the relations obtained in Step 1.
**Column 1 :** Cust_name from Borrower

$$\Pi_{customer\_name}\ (borrower)$$

**Column 2 :** Cust_name from Depositor

$$\Pi_{customer\_name}\ (depositor)$$

**Step 3 :** Identify the operator to be used. We need to find out the **names of customers** who are present in **both Borrower** table and **Depositor** table.
Hence, operator to be used—-> Intersection.

Final Query will be

$$\Pi_{customer\_name}\ (borrower) \cap \Pi_{customer\_name}\ (depositor)$$

Query 2. Retrieve the name and address of all employees who work for the 'Research' department.

RESEARCH_DEPT ← $\sigma_{Dname='Research'}$(DEPARTMENT)

RESEARCH_EMPS ← (RESEARCH_DEPT $\bowtie_{Dnumber=Dno}$EMPLOYEE)

RESULT ← $\pi_{Fname,\ Lname,\ Address}$(RESEARCH_EMPS)

As a single in-line expression, this query becomes:

$^{\pi}$Fname, Lname, Address $^{(\sigma}$Dname=‘Research’$^{(DEPARTMENT}$ ⋈ Dnumber=Dno$^{(EMPLOYEE))}$

This query could be specified in other ways; for example, the order of the JOIN and SELECT operations could be reversed, or the JOIN could be replaced by a NATURAL JOIN after renaming one of the join attributes to match the other join attribute name.

Query 3. For every project located in ‘Stafford’, list the project number, the controlling department number, and the department manager’s last name, address, and birth date.

STAFFORD_PROJS ← $\sigma_{Plocation='Stafford'}$(PROJECT)

CONTR_DEPTS ← (STAFFORD_PROJS ⋈$_{Dnum=Dnumber}$DEPARTMENT)

PROJ_DEPT_MGRS ← (CONTR_DEPTS ⋈$_{Mgr_ssn=Ssn}$EMPLOYEE)

RESULT ← $^{\pi}$Pnumber, Dnum, Lname, Address, Bdate$^{(PROJ_DEPT_MGRS)}$

In this example, we first select the projects located in Stafford, then join them with their controlling departments, and then join the result with the department man-agers. Finally, we apply a project operation on the desired attributes.

Query 4. Find the names of employees who work on *all* the projects controlled by department number 5.

DEPT5_PROJS ← $\rho_{(Pno)}(\pi_{Pnumber}(\sigma_{Dnum=5}$(PROJECT)))

EMP_PROJ ← $\rho_{(Ssn,\ Pno)}(\pi_{Essn,\ Pno}$(WORKS_ON))

RESULT_EMP_SSNS ← EMP_PROJ ÷ DEPT5_PROJS

RESULT ← $\pi_{Lname,\ Fname}$(RESULT_EMP_SSNS * EMPLOYEE)

In this query, we first create a table DEPT5_PROJS that contains the project numbers of all projects controlled by department 5. Then we create a table EMP_PROJ that holds (Ssn, Pno) tuples, and apply the division operation. Notice that we renamed the attributes so that they will be correctly used in the division operation. Finally, we join the result of the division, which holds only Ssn values, with the EMPLOYEE table to retrieve the desired attributes from EMPLOYEE.

Query 5. Make a list of project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

SMITHS(Essn) ← $\pi_{Ssn}$ ($\sigma_{Lname='Smith'}$(EMPLOYEE))

SMITH_WORKER_PROJS ← $\pi_{Pno}$(WORKS_ON * SMITHS)

MGRS ← $\pi_{Lname,\ Dnumber}$(EMPLOYEE $\bowtie_{Ssn=Mgr\_ssn}$ DEPARTMENT)

SMITH_MANAGED_DEPTS(Dnum) ← $\pi_{Dnumber}$ ($\sigma_{Lname='Smith'}$(MGRS))

SMITH_MGR_PROJS(Pno) ← $\pi_{Pnumber}$(SMITH_MANAGED_DEPTS * PROJECT)

RESULT ← (SMITH_WORKER_PROJS ∪ SMITH_MGR_PROJS)

In this query, we retrieved the project numbers for projects that involve an employee named Smith as a worker in SMITH_WORKER_PROJS. Then we retrieved the project numbers for projects that involve an employee named Smith as manager of the department that controls the project in SMITH_MGR_PROJS. Finally, we applied the UNION operation on SMITH_WORKER_PROJS and SMITH_MGR_PROJS. As a single in-line expression, this query becomes:

$\pi_{Pno}$ (WORKS_ON $\bowtie$ Essn=Ssn($\pi_{Ssn}$ ($\sigma_{Lname='Smith'}$(EMPLOYEE))) ∪ $\pi_{Pno}$

(($\pi_{Dnumber}$ ($\sigma_{Lname='Smith'}$($\pi_{Lname,\ Dnumber}$(EMPLOYEE)))

Ssn=Mgr_ssnDEPARTMENT)) $\bowtie$ Dnumber=DnumPROJECT)

Query 6. List the names of all employees with two or more dependents.

Strictly speaking, this query cannot be done in the *basic* (*original*) *relational algebra*. We have to use the AGGREGATE FUNCTION operation with the COUNT aggregate function. We assume that dependents of the *same* employee have *distinct* Dependent_name values.

$T1(\text{Ssn, No\_of\_dependents}) \leftarrow {}_{\text{Essn}}\Im_{\text{COUNT Dependent\_name}}(\text{DEPENDENT})$

$T2 \leftarrow \sigma_{\text{No\_of\_dependents}>2}(T1)$

$\text{RESULT} \leftarrow \pi_{\text{Lname, Fname}}(T2 * \text{EMPLOYEE})$

Query 7. Retrieve the names of employees who have no dependents.

This is an example of the type of query that uses the MINUS (SET DIFFERENCE) operation.

$\text{ALL\_EMPS} \leftarrow \pi_{\text{Ssn}}(\text{EMPLOYEE})$

$\text{EMPS\_WITH\_DEPS(Ssn)} \leftarrow \pi_{\text{Essn}}(\text{DEPENDENT})$

$\text{EMPS\_WITHOUT\_DEPS} \leftarrow (\text{ALL\_EMPS} - \text{EMPS\_WITH\_DEPS})$

$\text{RESULT} \leftarrow \pi_{\text{Lname, Fname}}(\text{EMPS\_WITHOUT\_DEPS} * \text{EMPLOYEE})$

We first retrieve a relation with all employee Ssns in ALL_EMPS. Then we create a table with the Ssns of employees who have at least one dependent in EMPS_WITH_DEPS. Then we apply the SET DIFFERENCE operation to retrieve employees Ssns with no dependents in EMPS_WITHOUT_DEPS, and finally join this with EMPLOYEE to retrieve the desired attributes. As a single in-line expression, this query becomes:

$\pi_{\text{Lname, Fname}}((\pi_{\text{Ssn}}(\text{EMPLOYEE}) - \rho_{\text{Ssn}}(\pi_{\text{Essn}}(\text{DEPENDENT}))) * \text{EMPLOYEE})$

Query 8. List the names of managers who have at least one dependent.

$\text{MGRS(Ssn)} \leftarrow \pi_{\text{Mgr\_ssn}}(\text{DEPARTMENT})$

$\text{EMPS\_WITH\_DEPS(Ssn)} \leftarrow \pi_{\text{Essn}}(\text{DEPENDENT})$

$\text{MGRS\_WITH\_DEPS} \leftarrow (\text{MGRS} \cap \text{EMPS\_WITH\_DEPS})$

$\text{RESULT} \leftarrow \pi_{\text{Lname, Fname}}(\text{MGRS\_WITH\_DEPS} * \text{EMPLOYEE})$

In this query, we retrieve the Ssns of managers in MGRS, and the Ssns of employees with at least one dependent in EMPS_WITH_DEPS, then we apply the SET INTERSECTION operation to get the Ssns of managers who have at least one dependent.

As we mentioned earlier, the same query can be specified in many different ways in relational algebra. In particular, the operations can often be applied in various orders. In addition, some operations can be used to replace others; for example, the INTERSECTION operation in Q8 can be replaced by a NATURAL JOIN. As an exercise, try to do each of these sample queries using different operations.[12] We showed how to write queries as single relational algebra expressions for queries Q2, Q5, and Q7. Try to write the remaining queries as single expressions. In Chapters 4 and 5 and in Sections 6.6 and 6.7, we show how these queries are written in other relational languages.